TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 3: Tokenization & Context Windows

Module 3 bridges raw character strings and the deep vector spaces of Transformers by deconstructing Tokenization and Context Window Economics. Every algorithmic difference, subword nuance, memory constraint, and pricing implication is detailed below.

3.1 Tokenization Deep Dive: BPE, WordPiece, and SentencePiece/Unigram

Before text enters an embedding matrix or an attention head, a tokenizer converts a raw text string into a sequence of discrete integer IDs. Three dominant subword algorithms form the foundation of modern LLMs: **Byte-Pair Encoding (BPE)**, **WordPiece**, and **SentencePiece (Unigram)**.

1. Byte-Pair Encoding (BPE)

Originally a 1994 data compression algorithm adapted for neural machine translation and later standardized across GPT, LLaMA, and Mistral.

- **The Training Algorithm:**
  1. Initialize the base vocabulary with raw characters or individual bytes (ensuring zero unknown tokens).
  2. Scan a massive pre-training corpus and count the frequency of all adjacent symbol pairs.
  3. Greedily merge the **most frequent pair** (e.g., *('e', 's') \to \text{es}*) into a new single vocabulary token.
  4. Repeat step 2–3 iteratively until reaching a pre-defined vocabulary size (e.g., 32,000 for LLaMA-1, 128,000 for LLaMA-3/4 or GPT-4).
- **Greedy Deterministic Encoding:** During inference, BPE applies the learned merge rules in the **exact chronological order** they were generated during training.

2. WordPiece (Used by BERT, DeBERTa)

- **Algorithmic Difference from BPE:** While BPE merges the *most frequent* pair, **WordPiece selects the pair that maximizes the likelihood of the training data according to a language model likelihood score**:

  $$\text{Score} = \frac{\text{Count}(XY)}{\text{Count}(X) \times \text{Count}(Y)}$$

  This favors pairs whose components frequently occur together relative to their individual frequencies.
- **Token Prefix Convention:** WordPiece uses a special continuation prefix (e.g., `##`) to mark subwords that appear in the middle or end of a word (e.g., *\text{"unaffable"} \to \texttt{["un", "##aff", "##able"]}*).

3. SentencePiece & Unigram Language Model (T5, ALBERT, LLaMA, Gemini)

Both BPE and WordPiece are bottom-up approaches. **Unigram** is a **top-down** probabilistic approach developed by Kudo (2018).

- **The Training Algorithm:** Start with an **oversized base vocabulary**, train a Unigram LM to estimate token probabilities, compute corpus loss reduction, and prune the bottom X% of tokens that contribute the least until reaching target vocabulary size.
- **Subword Regularization & Viterbi Decoding:** Because Unigram evaluates probabilistic subword segmentations, a single text string can be segmented in multiple ways (e.g., $\text{"unaffable"} \to \texttt{["un", "affable"]}$ vs $\texttt{["unaff", "able"]}$). During training, **Subword Regularization** samples alternative segmentations using Viterbi decoding, acting as a built-in regularizer.
- **SentencePiece Wrapper:** Treats input text as a raw continuous stream of bytes, treating spaces as a meta-character (_), making it completely language-agnostic.

Feature / Metric	Byte-Pair Encoding (BPE)	WordPiece	SentencePiece (Unigram)
Direction	Bottom-Up (Greedy frequent merges)	Bottom-Up (Likelihood-max merges)	Top-Down (Pruning from massive base)
Whitespace Handling	Splits on spaces/punctuation via pre-tokenizer	Whitespace splitting + `##` continuation markers	Treats whitespace as literal meta-character (_)
Ambiguity / Sampling	Deterministic greedy lookup	Deterministic greedy lookup	Probabilistic (supports regularization)
Frontier Models	GPT-2/3/4, LLaMA 1-4, Mistral, Qwen	BERT, RoBERTa, DeBERTa	T5, ALBERT, Gemini, multilingual models

3.2 What Is a "Token" Precisely? (And Why "Strawberry" Confuses LLMs)

- **The Precise Definition:** A token is an **atomic integer index** mapping into an embedding table matrix $E \in \mathbb{R}^{V \times d_{model}}$, where V is the vocabulary size (e.g., $128{,}000$). In English text, $1 \text{ token} \approx 4 \text{ characters}$ or 0.75 words. Common words occupy a single token; rare words, URLs, and UUIDs are split into multiple subwords.
- **Deconstructing the "Strawberry" Problem:** The viral phenomenon where LLMs struggle to count letter 'r's in "strawberry" stems from **token boundary blindness**:
  1. **Disrupted Continuity:** "strawberry" is tokenized into subwords such as $\texttt{["straw", "berry"]}$ or $\texttt{["st", "raw", "berry"]}$.
  2. **The Embedding Abstraction:** The model operates on atomic semantic vectors ("straw", "berry"), not raw character arrays $[s, t, r, a, w, b, e, r, r, y]$.

- 3. **Absence of Character-Level Gradients:** Pre-training rewards next-token prediction, not spelling character counts. Without explicit Chain-of-Thought or Python code execution, models rely on semantic associations leading to off-by-one errors.

3.3 Token Limits & Context Window Explained: Input vs. Output Budgets

The **Context Window** defines the maximum aggregate number of tokens—**Input (Prompt) + Output (Completion)** combined—residing in active memory during a single forward pass.

- **Shared Token Budget Architecture:** Operating as a single finite resource pool (*128\text{K}* tokens in GPT-4o/LLaMA-3; *1\text{M}–2\text{M}* in Gemini):

  > $$\text{Total Active Context} = \text{Tokens}_{\text{System}} + \text{Tokens}_{\text{History}} + \text{Tokens}_{\text{RAG}} + \text{Tokens}_{\text{Output}} \leq \text{Limit}$$

- **Implications:** If an 8K context prompt consumes 7,500 tokens, the model can generate a maximum of 500 completion tokens before hitting a hard context boundary.

3.4 Counting Tokens in Practice & Cost Implications

- **Counting Tokens:** Because tokenizers depend on specific vocabulary merge rules, you cannot use simple whitespace splitting (*\text{text.split()}*). Use programmatic libraries like OpenAI's *\text{tiktoken}* or Hugging Face's *\text{tokenizers}*.
- **Economic Cost Implications:** Commercial APIs bill strictly by token volume across two tiers:
  1. **Input Token Price (Prompt Cost):** Cheaper (*$0.15* to *$5.00* per 1M tokens) due to parallel prefill processing.
  2. **Output Token Price (Completion Cost):** Significantly more expensive (*$0.60* to *$15.00* per 1M tokens, *3\times* to *5\times* input cost) because sequential autoregressive generation is memory-bandwidth bound.
- **Production Optimizations:** Utilizing **Prompt Caching** (offering up to 50–90% discounts for static system prompts stored in KV cache) and stripping unnecessary whitespace from RAG chunks directly slashes monthly API billing overhead.